

PROJECT AND TEAM INFORMATION

Project Title

Campus Skill Swap Network

Student / Team Information

Team Name	SkillSync
Team Member 1 (Team Lead)	Mayank Rawat
Student ID	2510010445
Email	2510010445@geu.ac.in
Photo	
Team Member 2	Pranav Uniyal
Student ID	2510010601
Email	2510010601@geu.ac.in
Photo	
Team Member 3	Vaibhavi Saklani
Student ID	2510011544
Email	2510011544@geu.ac.in
Photo	
Team Member 4	Muskan
Student ID	2510010799
Email	2510010799@geu.ac.in
Photo	

Project Summary

Campus Skill Swap Network is a C++-based platform designed to help students learn from each other by exchanging skills within the campus community. Many students have valuable skills such as programming, designing, public speaking, video editing, music, or other technical and creative abilities, but they often do not know who around them can teach these skills.

The system creates a structured way for students to connect based on what they can teach and what they want to learn. Instead of depending only on friends, social media, or random groups, students can find suitable learning partners through a smart matching system. The project uses C++, Object-Oriented Programming, and Data Structures to manage student profiles, skills, requests, and rankings.

Our New Feature: Smart Multi-Way Skill Exchange

The main innovation in our project is Smart Multi-Way Skill Exchange. In a normal skill exchange, two students must directly match each other. For example, Student A teaches C++ to Student B, while Student B teaches Graphic Design to Student A. However, direct matching is not always possible.

Our system can identify a skill exchange chain. For example:

$$\text{Student A} \rightarrow \text{Student B} \rightarrow \text{Student C} \rightarrow \text{Student A}$$

This allows students to exchange skills even when a direct two-person match is unavailable. This feature can be implemented using Graph concepts in DSA, making the project more practical and technically strong.

Keywords

- Campus Skill Exchange
- Peer-to-Peer Learning
- Student Networking
- Skill Matching
- C++
- Data Structures
- Searching
- Sorting
- Student Profiles
- Online Learning
- Offline Learning
- Compatibility Matching
- Rating System
- Skill Sharing

PROPOSAL DESCRIPTION

Motivation

Learning has traditionally involved the sharing of knowledge among teachers, students, seniors, and peers. In college environments, students often help each other informally. For example, one student may be skilled in C++ programming, another may have expertise in graphic design, while others may have experience in video editing, public speaking, web development, or competitive programming. However, such knowledge sharing generally depends on personal friendships, classmates, WhatsApp groups, or other informal social connections.

As the number of students, skills, technical activities, workshops, and academic projects increases on a campus, finding the right person to learn from can become difficult. A student may want to learn a particular skill while another student on the same campus already possesses that skill, but both students may not know about each other. This creates a gap between students who want to learn and students who are willing to share their knowledge.

Existing online learning platforms provide courses, tutorials, and instructional content, but they mainly support one-way learning through recorded materials or instructors. They do not always provide a structured system for connecting students within the same campus for direct skill sharing and peer-to-peer learning.

Another limitation is that a simple two-person skill exchange may not always be possible. For example, Student A may know C++ and want to learn Python, while Student B knows Python but wants to learn Graphic Design. Although both students possess useful skills, their requirements do not match directly, making a two-person exchange impossible.

Our project aims to address this problem by developing a Campus Skill Swap Network that connects students based on the skills they can teach and the skills they want to learn. The system searches for suitable students, compares their skills and preferences, and recommends potential learning partners.

Multi-Way Skill Exchange:

A major feature of the proposed system is Multi-Way Skill Exchange. Instead of limiting skill exchange to a direct interaction between two students, the system can identify a chain of students through which skills can be exchanged indirectly.

Student A → Student B → Student C → Student A

Student A may teach a skill to Student B, Student B may help Student C, and Student C may provide the skill required by Student A. This multi-way exchange makes the system more flexible and increases the possibility of finding suitable learning opportunities among students.

The project also demonstrates how Data Structures and Algorithms can be applied to solve a real-world problem. Graphs can represent relationships between students and skills, searching techniques can identify suitable matches, hashing can support faster searches, and priority queues can rank the most suitable recommendations.

State of the Art / Current Solution

Existing methods of skill sharing mainly depend on personal connections, friends, classmates, college groups, or online learning platforms. While online platforms provide courses and tutorials, they do not provide an effective campus-level system for connecting students who can teach and learn from each other.

Our proposed solution is a **Campus Skill Swap Network** that provides a structured platform for students to discover suitable learning partners based on their skills, requirements, and preferences. A basic skill exchange can be represented as a direct relationship between two students. For example, Student A can teach C++ to Student B, while Student B can teach Python to Student A. Such an exchange can be represented as:

$$A \rightarrow B$$

However, direct matching is not always possible. To solve this limitation, our system supports **Multi-Way Skill Exchange**, where multiple students can participate in a connected skill-sharing chain:

$$A \rightarrow B \rightarrow C \rightarrow A$$

In this approach, each student can provide a skill required by another student, allowing useful exchanges even when a direct two-person match does not exist.

The proposed system considers multiple factors while finding suitable matches:

- Skills students can teach
- Skills students want to learn
- Learning preferences
- Online/Offline learning mode
- Location
- Student ratings
- Common technical interests
- Projects
- Workshops
- Competitions and Hackathons
- Previous learning experiences
- Multi-way skill exchange possibilities

The system represents possible skill exchange relationships using a **Graph**. Students can be represented as nodes, while possible skill connections can be represented as edges. **BFS** can be used to explore possible matches based on connection distance, while **DFS** can be used to explore deeper skill exchange paths.

The system can calculate a **compatibility score** by comparing student skills, learning requirements, and preferences. The generated matches can then be ranked using a **Priority Queue** so that the most suitable recommendations are presented first. **Hashing** can also be used to enable faster searching of students based on their skills.

For persistent data storage, **SQLite** can be used to store student profiles, skills, preferences, exchange requests, and feedback. When required, this information can be loaded into C++ data structures for matching, searching, and graph processing.

Thus, the proposed solution combines a campus-based skill-sharing platform with Data Structures and Algorithms to provide direct as well as multi-way learning opportunities among students.

Project Goals and Milestones

Major Goals of the Project:

1. Develop a C++-based Campus Skill Swap Network.
2. Create student profiles and manage student skills.
3. Allow students to add skills they can teach and skills they want to learn.
4. Search for suitable learning partners.
5. Implement direct skill matching and Multi-Way Skill Exchange.
6. Represent skill relationships using graphs.
7. Implement BFS and DFS for graph traversal.
8. Provide fast searching using hashing.
9. Calculate compatibility scores between students.
10. Rank suitable matches using a priority queue.
11. Manage skill exchange requests and feedback.
12. Store student and system information using SQLite.
13. Apply Object-Oriented Programming (OOP) and Data Structures and Algorithms (DSA) concepts to organize the complete system.

Initial Milestones:

1. **Milestone 1 – Requirement Analysis:** Identify system features, student information, skill requirements, and relevant DSA concepts.
2. **Milestone 2 – C++ and OOP Structure:** Create classes such as Student, Skill, Request, Match, and Graph.
3. **Milestone 3 – Student and Skill Management:** Implement student registration, profile creation, and skill management.
4. **Milestone 4 – Matching System:** Search and compare student skills to identify suitable learning partners.
5. **Milestone 5 – Graph Implementation:** Create skill relationships and implement BFS and DFS traversal.
6. **Milestone 6 – Multi-Way Skill Exchange:** Identify possible skill exchange chains between multiple students.
7. **Milestone 7 – Ranking System:** Calculate compatibility scores and rank suitable matches using a priority queue.
8. **Milestone 8 – Database Integration:** Store students, skills, requests, and feedback using SQLite.
9. **Milestone 9 – Testing and Final Integration:** Test all modules and integrate the complete system.

PROPOSED SOLUTION

Project Approach

The project will be developed using a modular C++ design, where each component performs a specific task. Student classes will manage information such as Student ID, skills offered, skills required, learning preferences, and ratings.

The system will search for students who possess the skills required by another student. If a suitable direct match is available, the system will recommend the corresponding student as a learning partner.

The main feature of the system is **Multi-Way Skill Exchange**. When a direct match is unavailable, the system will explore whether multiple students can form a skill exchange chain. For example, Student A can help Student B, Student B can help Student C, and Student C can provide the skill required by Student A.

Graphs will be used to represent possible skill relationships between students. **BFS** can be used to explore connections level by level, while **DFS** can be used to explore deeper paths and possible exchange cycles.

Compatibility scores will be calculated using factors such as skill matching, learning preferences, location, and ratings. Suitable candidates will then be ranked using a **Priority Queue**. Hashing through **unordered_map** can also be used to provide faster searching of students and skills.

SQLite will be used to store student profiles, skills, exchange requests, and feedback. The database will mainly handle persistent storage, while matching, graph processing, and other DSA operations will be performed in C++.

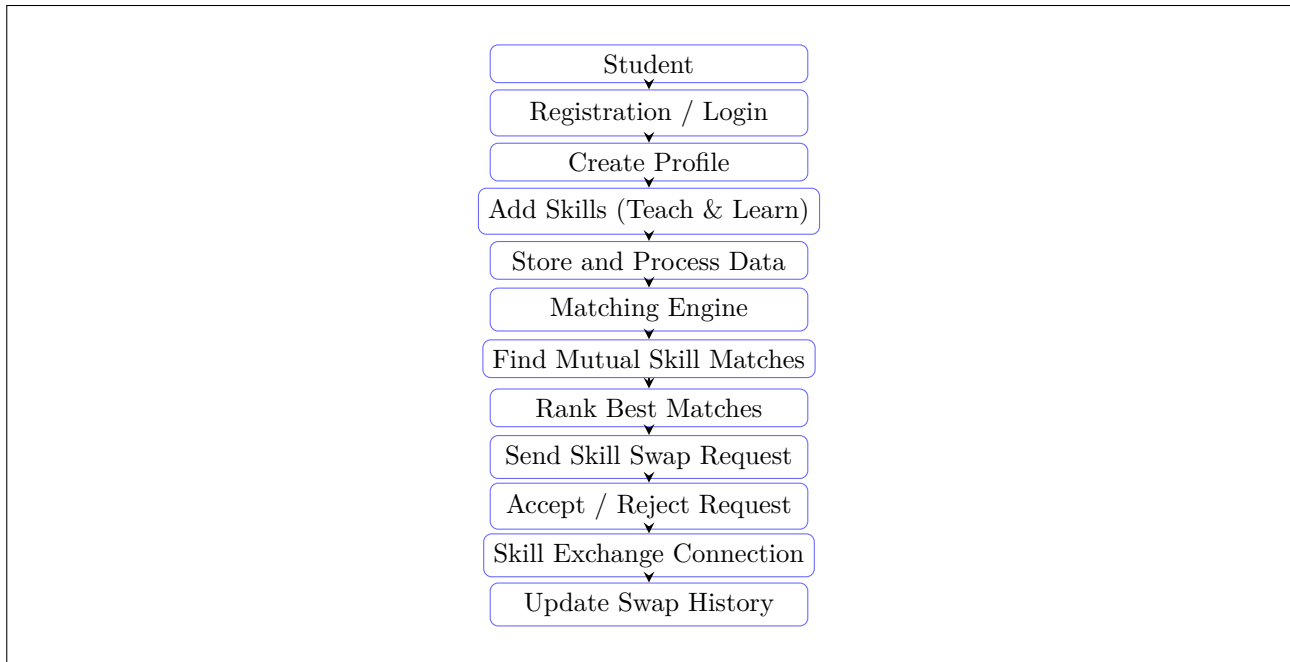
Overall System Flow:

Load Data → Create C++ Objects → Manage Skills → Build Graph
→ Search Matches → Apply BFS/DFS → Identify Direct or Multi-Way Exchange
→ Calculate Score → Rank Matches → Display Results

Key Features

- Student profile creation with skills and learning preferences.
- Add skills students can teach and want to learn.
- Skill-based matching between learners and suitable peer mentors.
- Multi-Way Skill Exchange through connected student skill chains.
- Graph-based representation of student skill relationships.
- BFS and DFS for exploring possible exchange paths.
- Compatibility scoring based on skills, preferences, location, ratings.
- Priority Queue for ranking the most suitable matches.
- Skill exchange requests with ratings and feedback.
- SQLite database for persistent student and exchange data.

System Flow



SYSTEM PROCESS AND PROJECT OUTCOME

System Flow Description

System Flow Description:

1. **Student Registration and Login:** The student registers and creates a profile in the system.
2. **Skill Entry:** The student adds skills they can teach and skills they want to learn.
3. **Skill Data Processing:** The system stores and organizes student and skill information using appropriate data structures.
4. **Skill Matching:** The Matching Engine searches for students whose offered skills match the required skills of others.
5. **Mutual Match Detection:** The system identifies mutual exchanges where students can teach each other their required skills.
6. **Match Ranking:** Suitable matches are ranked based on skill compatibility using a Priority Queue.
7. **Skill Swap Request:** The student sends a skill swap request to a recommended student.
8. **Request Processing:** The receiving student can accept or reject the request. Requests are managed using a Queue.
9. **Swap Connection:** If the request is accepted, both students are connected for skill exchange.
10. **Data Storage and Update:** The system updates student profiles, skills, requests, and swap history.

Project Outcome / Deliverables

The project will provide a simple platform for students to find suitable peers for learning and exchanging skills. It will make skill discovery and matching easier while demonstrating the use of C++, OOP, and DSA concepts in a practical application.

Expected Outcomes

- A working Campus Skill Swap Network in C++.
- Easy student registration and profile management.
- Skill-based matching between learners and mentors.
- Ranking of suitable matches using DSA concepts.
- Skill exchange request and feedback system.
- Practical implementation of C++, OOP, and DSA concepts.

Project Deliverables

- Complete C++ source code for the Campus Skill Swap Network.
- Student registration and profile management module.
- Skill matching and compatibility ranking module.
- Request, skill exchange, and feedback module.
- Final project report and documentation.

Assumptions

The following assumptions are made for the development of the system:

1. Every student has a unique Student ID.
2. Students belong to the same campus.
3. Students provide correct information about their skills.
4. A student can have multiple skills.
5. A student can be both a learner and a mentor.
6. The compatibility score depends on the available information.
7. A higher score indicates a better match.
8. A recommended student may reject a skill exchange request.
9. Multi-Way Exchange is possible only when a valid exchange path exists.
10. SQLite is used for persistent storage.
11. Graph processing is performed in C++.
12. The project is an academic prototype for campus-level use.
13. The recommendation system is rule-based and does not use machine learning.

Technology Stack

Programming Language	C / C++
Data Structures	Arrays, Linked Lists, Queues
Object-Oriented Concepts	Classes, Objects, Encapsulation, STL
Development Environment	Visual Studio Code
Version Control	Git / GitHub

REFERENCES

1. Bjarne Stroustrup, *Programming: Principles and Practice Using C++*, Addison-Wesley.
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, Pearson.
3. Content about queues, linked lists, priority-based processing, and algo based order management.